

Agent Mesh SRE — Implementation Checklist v2.1

Document version: 2.1 **Application version:** Current (surajcsibm fork, May 2026) **Last updated:** May 2026

Table of Contents

- Part A: File Reference
 - Part B: Environment Setup
 - Part C: Implementation Phases
 - Part D: Verification
 - Part E: Key Patterns
 - Part F: Dependencies
-

Part A: File Reference

Core source files

File	Purpose
src/components/Dashboard.tsx	Entire dashboard UI — scenarios, canvas host, right panel, history bar, topics panel
src/components/AgentCanvas.tsx	React Flow canvas with circular agent nodes and sub-agent circles
src/components/useMeshStream.ts	Custom hook — useReducer state, simulation control, topic actions
src/lib/client-sim.ts	Deterministic scenario simulation engine (all 10 scenarios)
src/lib/types.ts	Shared TypeScript interfaces (AgentState, AuditRecord, KafkaTopic, etc.)
src/lib/emailer.ts	Nodemailer wrapper + HTML email template
src/app/dashboard/page.tsx	Server component wrapper (auth guard)
src/app/login/page.tsx	Login/register page
src/app/api/auth/[...nextauth]/route.ts	NextAuth handler (Google + Credentials)
src/app/api/send-email/route.ts	SMTP email dispatch route
src/middleware.ts	Auth middleware — protects /dashboard
next.config.mjs	ignoreBuildErrors: true + image domains
tailwind.config.ts	Tailwind config (custom animations:

`.env.local.example`
`FORK-COMPARISON.md`

slideDown, slideInRight, pulse)
Environment variable template
Fork analysis — surajcsibm vs
aswinayyolath PR #1

Files NOT present in this fork (upstream only)

File	Reason absent
<code>src/lib/mesh.ts</code>	Replaced by <code>client-sim.ts</code>
<code>src/store/*.ts</code>	No Zustand — replaced by <code>useReducer</code>
<code>src/lib/sse-client.ts</code>	No SSE streaming — simulation is local
<code>src/components/ScenarioPanel.tsx</code>	Merged into <code>Dashboard.tsx</code>
<code>src/components/SetupWizard.tsx</code>	Removed — no infrastructure to configure
<code>src/components/VisualBuilder.tsx</code>	Removed — not in scope for this fork

Part B: Environment Setup

Required environment variables

Authentication

`GOOGLE_CLIENT_ID=<your-google-client-id>`
`GOOGLE_CLIENT_SECRET=<your-google-client-secret>`
`NEXTAUTH_SECRET=<random-32-char-string>`
`NEXTAUTH_URL=http://localhost:3000`

Email (Nodemailer / SMTP)

`SMTP_HOST=smtp.gmail.com`
`SMTP_PORT=587`
`SMTP_USER=your@email.com`
`SMTP_PASS=<app-password>`
`SMTP_FROM=your@email.com`
`NOTIFY_TO=recipient@email.com`

Pre-requisites checklist

- ☐ Node.js 18+ installed
 - ☐ npm 9+ installed
 - ☐ Google Cloud project created with OAuth 2.0 credentials
 - ☐ OAuth redirect URI set to `{NEXTAUTH_URL}/api/auth/callback/google`
 - ☐ SMTP credentials available (Gmail app password or other provider)
 - ☐ Git installed
-

Part C: Implementation Phases

Phase 1 — Project Scaffold

- ☐ `git clone https://github.com/surajcsibm/agent-mesh-sre.git`
- ☐ `npm install`
- ☐ Copy `.env.local.example` to `.env.local` and fill all values
- ☐ Verify `npm run dev` starts without errors

Phase 2 — Authentication

- ☐ `src/app/api/auth/[...nextauth]/route.ts` — Google + Credentials providers wired
- ☐ `src/middleware.ts` — protects `/dashboard`, allows `/login` and `/api/auth/*`
- ☐ `src/app/login/page.tsx` — login and register forms rendered
- ☐ Forgot-password email flow tested (SMTP must be configured)
- ☐ Verify: navigate to `/dashboard` unauthenticated → redirect to `/login`
- ☐ Verify: Google OAuth sign-in completes and returns to `/dashboard`
- ☐ Verify: email/password register + login cycle works

Phase 3 — Simulation Engine

- ☐ `src/lib/types.ts` — `AgentState`, `AuditRecord`, `MCPToolCall`, `ApprovalRequest`, `EmailSummaryData`, `KafkaTopic` defined
- ☐ `src/lib/client-sim.ts` — all 10 scenarios implemented as async functions
- ☐ `src/components/useMeshStream.ts` — `useReducer` with full `MeshState`, all action handlers, `runScenario` integration
- ☐ Verify: clicking any scenario card triggers simulation (agents animate)
- ☐ Verify: AWAITING phase surfaces approval card
- ☐ Verify: approve/reject continues the MRAL loop to completion

Phase 4 — Agent Canvas

- ☐ `src/components/AgentCanvas.tsx` — dynamically imported, SSR disabled
- ☐ `AgentNode` component — 200 px circle, status label, Kill/Restart buttons
- ☐ `BrokerNode` component — 160 px circle, navy theme
- ☐ `SubAgentNode` component — 90 px dashed circle, ephemeral MRAL phase circles
- ☐ Node positions set for all 5 primary nodes + sub-agent position
- ☐ Accent colours: INTAKE emerald, MONITOR violet, WRITER cyan, NOTIFY orange
- ☐ Kill button: `e.stopPropagation()` + `pointerEvents: "all"` on button style
- ☐ Confirm kill popup: same click-propagation fix applied
- ☐ Canvas + / - height controls (80 px step, 380–960 px range)
- ☐ Verify: Kill button clickable inside circular node

- ☐ Verify: Sub-agent REASON circle appears during MONITOR reasoning phase
- ☐ Verify: Sub-agent ACT circle appears during MONITOR acting phase
- ☐ Verify: Sub-agent LEARN circle appears during MONITOR learning phase

Phase 5 — Dashboard UI

- ☐ Arctic Clean theme applied (navy nav, emerald accent, #f0f4f8 bg)
- ☐ Left sidebar: pinned 4 scenarios + scrollable 6 extra scenarios
- ☐ Centre: AgentCanvas with height controls
- ☐ Right sidebar: Live Feed, Topics, Audit tabs
- ☐ Live Activity Banner slides in/out based on agent status
- ☐ Approval card rendered when `state.approvalRequest` is set
- ☐ Scenario End Modal shown when `state.emailSummary` is set
- ☐ Toast stack for info/success/warning/error notifications

Phase 6 — Topics Management

- ☐ Topics panel in left sidebar renders `KafkaTopic[ ]` from component state
- ☐ Each topic card shows: name, partitions, lag with ▲/▼ trend, msg/s, health badge
- ☐ Health badge: green (healthy), amber (degraded), red (critical)
- ☐ **SCENARIO_AUTO_TOPICS map defined** — all 10 scenario IDs mapped to their relevant topics with accurate initial lag and health values
- ☐ **handleTrigger wrapper** — upserts scenario topics into state before calling `trigger()`. If topic name exists, updates lag/health/msgPerSec in place; if new, inserts at front of list
- ☐ **Auto-create verified** — clicking a scenario immediately populates its topics in the panel with scenario-appropriate health status
- ☐ Heal Topic button triggers dedicated MRAL cycle
- ☐ Create Topic form — name, partitions, replication, retention inputs, duplicate-name validation
- ☐ Copy Topic — clones with -copy suffix, resets lag to zero
- ☐ **TopicsPanel pagination** — accepts `visibleCount` and `onShowMore` props; renders `sorted.slice(0, visibleCount)`
- ☐ **“+ N more topics ↓” button** — appears when `remaining > 0`; calls `onShowMore` which increments `topicsVisible` by 20
- ☐ Panel starts at 20 visible; each More click adds 20 more
- ☐ Full list is scrollable (`maxHeight: 420px, overflow-y: auto`)
- ☐ Background topic auto-generation — new random topic every 18–40 s
- ☐ Verify: Create Topic with duplicate name shows error
- ☐ Verify: Heal Topic triggers MRAL cycle visible on canvas
- ☐ Verify: running a scenario auto-populates its topics before agents start

- ☐ Verify: More button loads additional topics, count increments by 20

Phase 7 — Live Feed & Audit Log

- ☐ Live Feed tab shows audit events prepended (newest first)
- ☐ Events colour-coded by type
- ☐ Audit tab shows same events in table format
- ☐ Live Feed events captured into `EmailSummaryData.liveEvents` at scenario end
- ☐ Hydration from `auditLogRef` when `liveEvents` not explicitly populated by sim

Phase 8 — Scenario History Persistence

- ☐ `summaryHistory` state initialised with `[]` (not from `localStorage`)
- ☐ `useEffect([], [])` loads from `localStorage` after mount; sets `historyMounted = true`
- ☐ Save effect guarded by `historyMounted` to prevent early overwrite
- ☐ `lastAddedSummaryRef` prevents duplicate capture of same `emailSummary` object
- ☐ History bar rendered at bottom of dashboard (below canvas)
- ☐ Each history entry clickable → opens Summary Modal with live events table
- ☐ Clear History button wipes `localStorage` and state
- ☐ History capped at 30 entries (oldest dropped)
- ☐ Verify: refresh page → history entries still present
- ☐ Verify: run same scenario twice → two separate entries appear

Phase 9 — Email Notifications

- ☐ `src/lib/emailer.ts` — Nodemailer transport created from env vars
- ☐ HTML email template includes scenario summary, lag delta, reasoning, action, lesson
- ☐ **Live Feed Events table** included in email body
- ☐ `/api/send-email` POST route calls `emailer.ts` and returns status
- ☐ Dashboard posts to `/api/send-email` when NOTIFY agent fires
- ☐ Email error shown in Scenario End Modal if send fails
- ☐ Verify: complete a scenario with SMTP configured → email received
- ☐ Verify: email body contains live feed events table

Phase 10 — Fork Documentation

- ☐ `FORK-COMPARISON.md` present in repository root
- ☐ File documents architectural differences vs `aswinayyolath PR #1`
- ☐ PR branch `pr/addendum-fork-comparison` pushed to `surajcsibm/agent-mesh-sre`
- ☐ Pull request raised against `aswinayyolath/agent-mesh-sre:main` using `FORK-COMPARISON.md` as description

Part D: Verification

Smoke tests (run after `npm run dev`)

Test	Expected result
Navigate to /	Redirect to /dashboard or /login
Sign in with Google	Redirected to dashboard
Click “Consumer Lag Spike”	<code>ops.kafka.metrics.v1</code> (critical) appears in Topics panel immediately
Agents animate	Live Activity Banner appears
MONITOR reaches AWAITING	Approval card appears in centre
Click Approve	MRAL loop continues, completes
Scenario End Modal	Shows summary with live events table
Close modal	Entry appears in history bar
Refresh page	History entry persists
Click history entry	Summary modal opens with live events
Click Kill on INTAKE	Agent node shows Killed state
Click Restart	Agent returns to Idle
Click + canvas button	Canvas grows by 80 px
Click Topics tab	Topic list renders with health badges
Click Heal Topic	MRAL cycle starts for that topic
Create Topic	New topic appears in list
Duplicate topic name	Error shown, topic not added
Click More (topics)	Next 20 topics load, count label updates
Run all 10 scenarios	Each seeds its own unique topics
<code>npm run build</code>	Build completes (<code>ignoreBuildErrors: true</code>)

Common issues and fixes

Issue	Cause	Fix
Hydration error on ScenarioHistoryBar	<code>localStorage</code> read in <code>useState</code> initialiser	Initialise with <code>[]</code> ; load in <code>useEffect</code>
Kill button not clickable	React Flow consuming pointer events	Add <code>e.stopPropagation()</code> + <code>pointerEvents: "all"</code>
History shows only latest entry	<code>auditLog</code> in effect deps causes re-runs	Use <code>auditLogRef</code> ; use <code>object-ref dedup</code>
Topics not appearing on scenario start	<code>handleTrigger</code> not wired or <code>SCENARIO_AUTO_TOPICS</code> missing	Check all 10 scenarios have entries in the map

More button missing	entry visibleCount/onShowMore props not passed to TopicsPanel	Pass topicsVisible and setTopicsVisible from Dashboard
Sub-agent circle doesn't appear	agentMap missing from edges useMemo deps	Add agentMap to dependency array
Build fails on type error	React Flow v12 type mismatch	ignoreBuildErrors: true in next.config.mjs
Email not sent	SMTP misconfiguration	Check env vars; use Gmail app password

Part E: Key Patterns

Scenario auto-create topics

Define SCENARIO_AUTO_TOPICS as a `Record<string, Omit<KafkaTopic, "id">[]>`. Wrap `trigger()` in `handleTrigger()` which iterates the relevant topics and upserts them into `setTopics()` state before calling `trigger(scenarioId)`. Use `findIndex` to decide insert vs update.

TopicsPanel pagination

Pass `visibleCount: number` and `onShowMore: () => void` as props. Render `sorted.slice(0, visibleCount)`. Show a "More" button when `sorted.length > visibleCount`. Parent manages `topicsVisible` state and increments by 20 per click.

Client-side simulation

All scenarios run as async functions in `client-sim.ts`. Use `await delay(ms)` between steps. Use `dispatch({ type: "AGENT_UPDATE", ... })` to update agent state. Use `await waitForDecision(getState)` to pause for human approval.

Hydration-safe localStorage

Always init `useState` with a static value. Load from `localStorage` in `useEffect`. Guard save effects with a mounted boolean so the empty initial state does not overwrite stored data.

ReactFlow click propagation

Any interactive element inside a ReactFlow custom node must call `e.stopPropagation()` on its click handler and have `pointerEvents: "all"` in its inline style.

Part F: Dependencies

Runtime dependencies

Package	Version	Purpose
next	14.x	Framework
react, react-dom	18.x	UI
@xyflow/react	12.x	Agent canvas
next-auth	4.x	Authentication
nodemailer	6.x	SMTP email
clsx	2.x	Conditional classNames
tailwindcss	3.x	Utility CSS

Notable absence from upstream

The following packages from the upstream reference implementation are **not** required in this fork: zustand, kafkajs, ioredis, bullmq. The simulation engine replaces all of these.